

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнила: Я. Р. Тарабрина
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: познакомиться с основами трёхмерной графики: построением простых 3D-объектов, их проекцией на плоскость экрана, матрицами перспективной и ортогографической проекций и аффинными преобразованиями.

Вариант 10: правильный додекаэдр.

Программа написана на C++20 с использованием Vulkan, GLFW и Dear ImGui на основе стартового проекта. Она отрисовывает додекаэдр в реальном времени и позволяет во время работы менять проекцию, трансформации и параметры анимации объектов.

Выполнены все дополнительные задания:

1. переключение ортогографической и перспективной проекций в интерфейсе;
2. изменение позиции, поворота и масштаба фигуры;
3. движение и вращение фигуры по сложной траектории с паузой, регулировкой скорости и параметров траектории;
4. выбор цвета фигуры;
5. процедурные цвета вершин, умножаемые на выбранный цвет;
6. отрисовка нескольких объектов, у каждого из которых свой набор дескрипторов.

Исходный код: <https://github.com/batsuperflash/cg-labs>.

Метод решения

Построение додекаэдра

Правильный додекаэдр имеет 20 вершин, 30 рёбер и 12 граней — правильных пятиугольников. Его вершины выражаются через золотое сечение $\varphi = (1 + \sqrt{5})/2$:

$$(\pm 1, \pm 1, \pm 1), \quad (0, \pm \varphi^{-1}, \pm \varphi), \quad (\pm \varphi^{-1}, \pm \varphi, 0), \quad (\pm \varphi, 0, \pm \varphi^{-1}).$$

Все вершины лежат на сфере радиуса $\sqrt{3}$, поэтому после деления на $\sqrt{3}$ фигура вписана в единичную сферу.

Нормали граней направлены на вершины двойственного икосаэдра: $(0, \pm \varphi, \pm 1)$ и их циклические перестановки. Для каждой нормали $\mathbf{n}$

гранью являются пять вершин с наибольшим значением $\langle \mathbf{v}, \mathbf{n} \rangle$. Вершины грани упорядочиваются по углу в базисе $(\mathbf{u}, \mathbf{w})$, где $\mathbf{u}$ направлен от центра грани к одной из её вершин, а $\mathbf{w} = \mathbf{n} \times \mathbf{u}$. Такой порядок обходит грань против часовой стрелки, если смотреть снаружи. Каждый пятиугольник разбивается веером на три треугольника: всего получается 36 треугольников и 108 индексов.

Геометрия проверяется автоматическими тестами. Все 30 рёбер имеют одинаковую длину, выполняется формула Эйлера $V - E + F = 20 - 30 + 12 = 2$, а каждое ребро треугольников встречается ровно дважды в противоположных направлениях, то есть поверхность замкнута и все треугольники ориентированы одинаково.

Аффинные преобразования

Точки записываются в однородных координатах $(x, y, z, 1)$, а преобразования — матрицами 4×4 . Перенос на вектор $\mathbf{t}$ и масштабирование с коэффициентами $\mathbf{s}$:

$$T(\mathbf{t}) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S(\mathbf{s}) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Поворот на угол γ вокруг оси Z (матрицы R_x и R_y строятся аналогично):

$$R_z(\gamma) = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 & 0 \\ \sin \gamma & \cos \gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Матрица модели объекта:

$$M = T(\mathbf{p}(t) + \mathbf{c}) R_y(\theta) R_x(\theta/2) R_z(\gamma) R_y(\beta) R_x(\alpha) S(\mathbf{s}),$$

где $\mathbf{c}$ — позиция из интерфейса, α, β, γ — углы поворота, $\mathbf{s}$ — масштаб по осям, θ — угол собственного вращения, $\mathbf{p}(t)$ — точка траектории. Преобразования применяются справа налево: сначала масштаб в локальных осях фигуры, затем поворот, собственное вращение и перенос. Если масштабировать после поворота, неравномерное растяжение перекосит повернутую фигуру. Отрицательный масштаб в интерфейсе запрещён: он меняет направление обхода треугольников, и отсечение задних граней скрыло бы лицевые.

Камера и проекции

Камера находится в точке $\mathbf{e} = (0,0,d)$ и смотрит в начало координат. Матрица вида строится из базиса камеры $\mathbf{f} = \text{normalize}(\mathbf{target} - \mathbf{e})$, $\mathbf{r} = \text{normalize}(\mathbf{f} \times \mathbf{up})$, $\mathbf{u} = \mathbf{r} \times \mathbf{f}$:

$$V = \begin{pmatrix} r_x & r_y & r_z & -\langle \mathbf{r}, \mathbf{e} \rangle \\ u_x & u_y & u_z & -\langle \mathbf{u}, \mathbf{e} \rangle \\ -f_x & -f_y & -f_z & \langle \mathbf{f}, \mathbf{e} \rangle \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

В Vulkan, в отличие от OpenGL, ось Y пространства отсечения направлена вниз, а глубина после деления на w лежит в диапазоне $[0,1]$. Матрицы проекций выведены под эти соглашения. Перспективная проекция с углом обзора fov , соотношением сторон a и плоскостями отсечения z_n и z_f :

$$P_{persp} = \begin{pmatrix} \frac{k}{a} & 0 & 0 & 0 \\ 0 & -k & 0 & 0 \\ 0 & 0 & \frac{z_f}{z_n - z_f} & \frac{z_n z_f}{z_n - z_f} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \quad k = \frac{1}{\tan(fov/2)}.$$

Последняя строка даёт $w = -z$, поэтому после деления на w удалённые объекты уменьшаются. Точка на глубине $z = -z_n$ переходит в 0, точка на глубине $z = -z_f$ — в 1, а минус во второй строке переворачивает ось Y .

Ортографическая проекция переводит параллелепипед $[l,r] \times [b,t] \times [-z_f, -z_n]$ в объём отсечения:

$$P_{ortho} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & -\frac{2}{t-b} & 0 & \frac{t+b}{t-b} \\ 0 & 0 & -\frac{1}{z_f - z_n} & -\frac{z_n}{z_f - z_n} \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Высота видимого объёма выбрана равной $h = 2d \tan(fov/2)$, ширина — $h \cdot a$. Ровно такую высоту перспективная камера видит в плоскости $z = 0$, поэтому при переключении проекций размер фигуры в

центре сцены не меняется. Все матрицы в тестах сверяются с функциями библиотеки GLM (`perspectiveRH_ZO` и `orthoRH_ZO` с отражённой осью Y , `lookAtRH`).

Траектория и анимация

Фигура движется по трёхмерной фигуре Лиссажу:

$$\mathbf{p}(t) = R(\sin(at + \delta), \sin(bt), \sin(ct)),$$

где R — радиус, a, b, c — целые частоты, δ — сдвиг фазы (по умолчанию $R = 2$, частоты 1, 2 и 3, $\delta = \pi/2$). Параметр t накапливается каждый кадр: $t \leftarrow t + v \Delta t$, где v — скорость, Δt — время кадра. Поэтому пауза и смена скорости не вызывают скачков. При целых частотах траектория периодична с периодом 2π , и t хранится по модулю 2π , чтобы не терять точность чисел с плавающей точкой. Угол собственного вращения θ накапливается так же. Объекты стартуют в разных точках траектории: объект с номером i сдвинут по параметру на $2\pi i/8$.

Цвет

Цвет вершины вычисляется из её положения: **color** = $0.5 \cdot \mathbf{position} + 0.5$, так что противоположные стороны фигуры окрашены в дополнительные цвета. Итоговый цвет — произведение цвета вершины на цвет из интерфейса, а при выключенных цветах вершин — только цвет из интерфейса.

`Swarchain` создаётся в формате `B8G8R8A8_SRGB`: шейдер выводит линейные значения, а при записи в изображение они переводятся в `sRGB`. Поэтому процедурные цвета вершин переводятся из `sRGB` в линейное пространство при генерации, иначе градиент выглядел бы блёклым. Цвет из интерфейса передаётся без перевода: `ImGui` рисует образец цвета в тот же `swarchain` без перевода, и только так фигура совпадает с образцом.

Используемые объекты Vulkan

- **VkBuffer** и **VmaAllocation** (Vulkan Memory Allocator): вершинный буфер (20 вершин с позицией и цветом), индексный буфер (108 индексов `uint16`) и по одному `uniform buffer` на объект. `Uniform buffer` хранит матрицы M, V, P , цвет и флаги в раскладке `std140` (224 байта). Буферы находятся в памяти, доступной процессору, и постоянно отображены в его адресное пространство.

- **VkShaderModule:** вершинный и фрагментный шейдеры на GLSL, скомпилированные в SPIR-V утилитой `glslc`. Вершинный шейдер вычисляет положение *PVMx* и цвет, фрагментный выводит цвет.
- **VkDescriptorSetLayout:** одна привязка — uniform buffer для вершинного и фрагментного шейдеров.
- **VkDescriptorPool и VkDescriptorSet:** восемь наборов дескрипторов, по одному на объект, каждый указывает на uniform buffer своего объекта.
- **VkPipelineLayout и VkPipeline:** графический конвейер с атрибутами вершин (позиция и цвет), топологией `TRIANGLE_LIST` (MoltenVK на macOS не поддерживает веер треугольников), отсечением задних граней, тестом глубины `LESS` и динамическими viewport и scissor, поэтому при изменении размера окна конвейер не пересоздаётся.
- **VkRenderPass и VkFramebuffer** из стартового проекта: изображение `swarchain` и буфер глубины, которые очищаются в начале каждого кадра.
- **VkCommandBuffer:** каждый кадр в него записываются начало прохода рендеринга, viewport и scissor, привязка конвейера и буферов, а затем для каждого объекта привязка его набора дескрипторов и вызов `vkCmdDrawIndexed`.

Стартовый проект держит в работе один кадр: перед подготовкой нового кадра он ждёт fence предыдущего. Поэтому uniform buffers записываются в функции `render()` после этого ожидания, а не в `update()`, которая выполняется раньше. Так GPU никогда не читает буфер, пока процессор его перезаписывает.

Несколько объектов

Команды из командного буфера выполняются на GPU после того, как запись закончена. Если бы все объекты брали данные из одного uniform buffer, перезапись между вызовами отрисовки ничего бы не дала: все вызовы прочитали бы последние записанные данные. Поэтому у каждого объекта свой uniform buffer и свой набор дескрипторов, и перед отрисовкой объекта функцией `vkCmdBindDescriptorSets` привязывается его набор. Буферы и наборы создаются при запуске сразу для восьми объектов, поэтому во время работы ничего не создаётся и не удаляется.

Возможны и другие способы передать данные нескольких объектов:

- push constants: матрица модели и цвет передаются прямо в команде (гарантированный объём — 128 байт), а матрицы вида и проекции остаются в общем uniform buffer;
- dynamic uniform buffer: один набор дескрипторов и один буфер, а смещение данных объекта задаётся при привязке набора;
- instancing: один вызов отрисовки на все объекты, а их данные хранятся в буфере экземпляров.

Результаты

На рисунке 1 показан основной вариант: додекаэдр в перспективной проекции с процедурными цветами вершин. Фигура движется по траектории и вращается. Слева — панель управления с разделами для камеры, объектов, трансформаций, анимации и цвета.

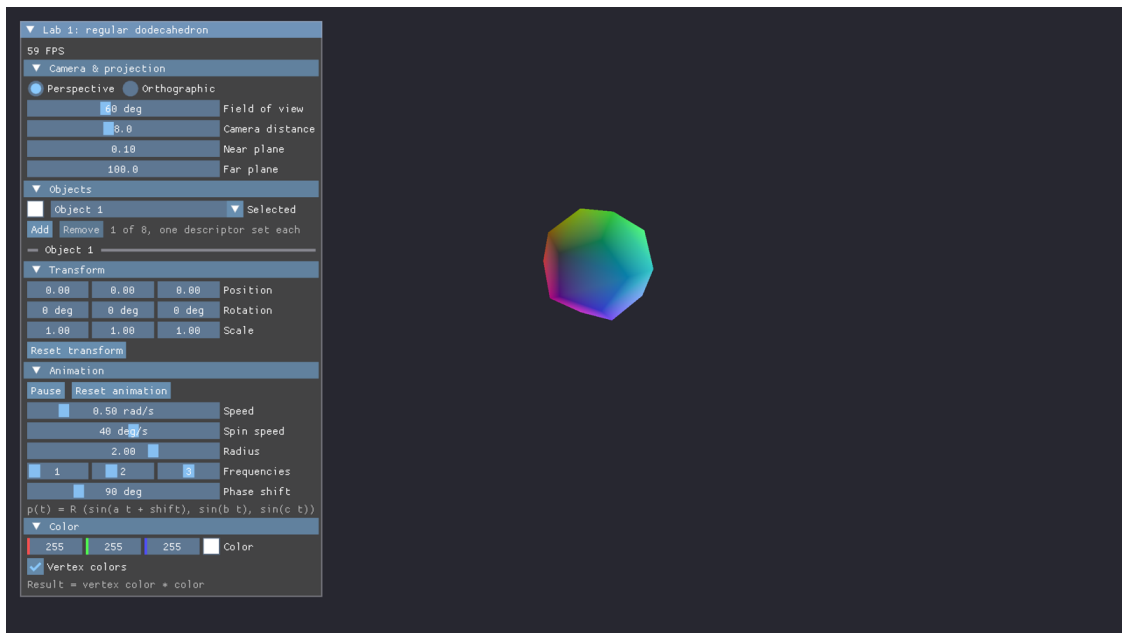


Рисунок 1 — Основной вариант в перспективной проекции

На рисунке 2 включена ортогографическая проекция. Фигура растянута вдвое по оси X , сжата по оси Y и повёрнута, анимация поставлена на паузу. Параллельные рёбра остаются параллельными, перспективного сокращения нет.

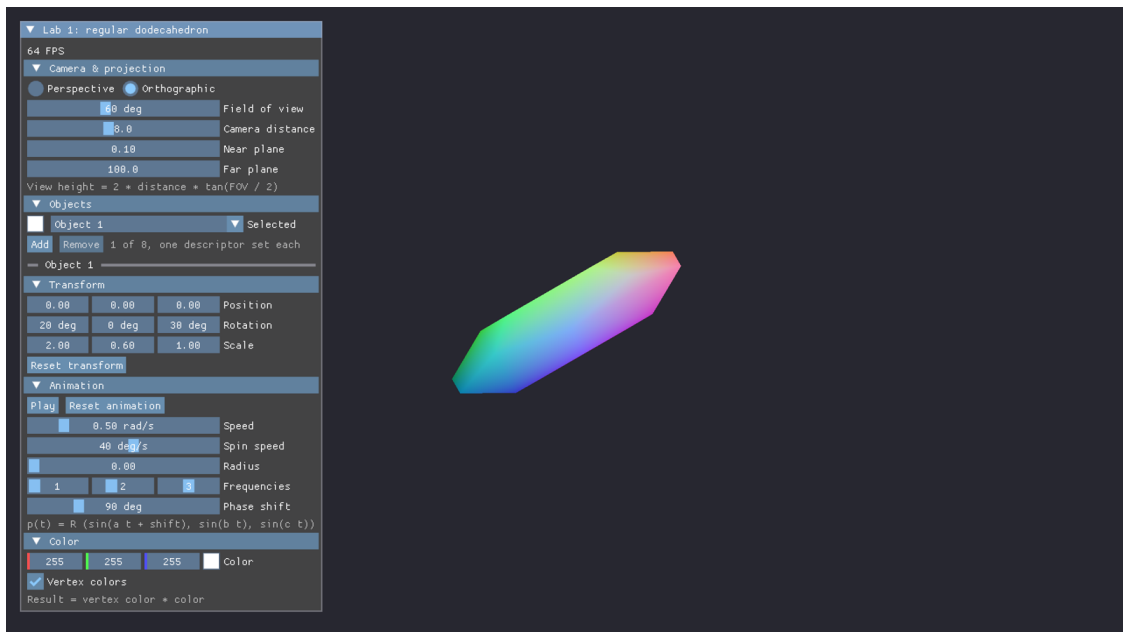


Рисунок 2 — Ортогографическая проекция и трансформации фигуры

На рисунке 3 на сцене восемь объектов. Все они движутся по одной траектории, но стартуют из разных её точек, и у каждого свой оттенок, который умножается на цвета вершин. Ближние фигуры закрывают дальние благодаря тесту глубины.

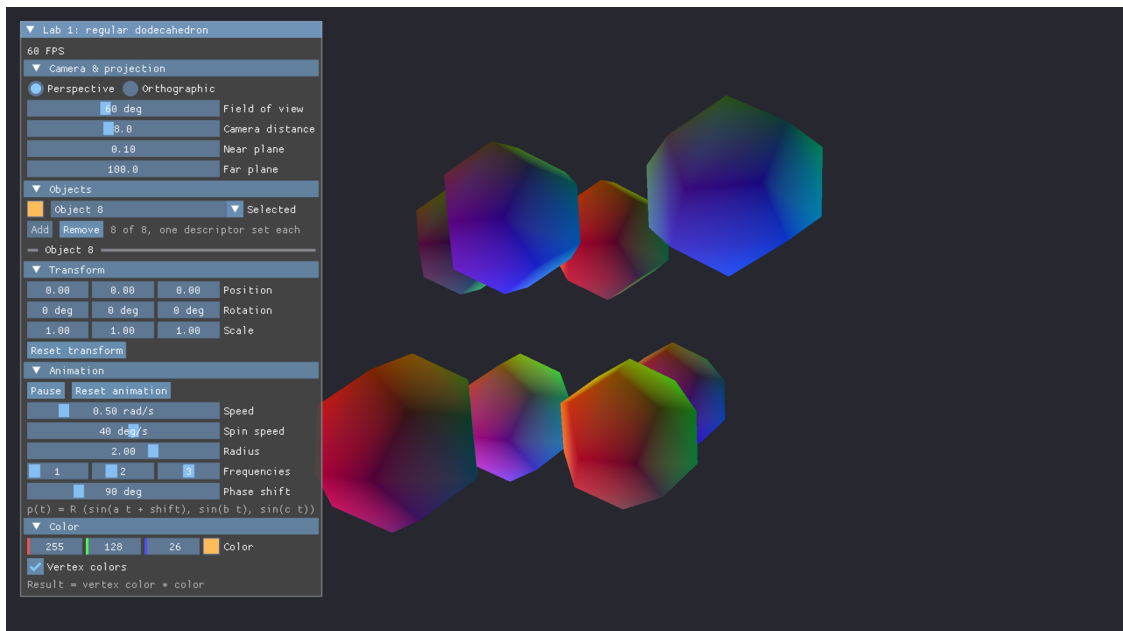


Рисунок 3 — Восемь объектов, у каждого свой набор дескрипторов

Автоматические тесты (147 проверок) проходят, а слой валидации Vulkan не выдаёт сообщений ни во время работы, ни при завершении программы.

Выводы

Я научилась строить многогранник по координатам вершин, работать с однородными координатами и аффинными преобразованиями, выводить матрицы перспективной и ортографической проекций под соглашения Vulkan, передавать данные объектов через uniform buffers и наборы дескрипторов, записывать команды отрисовки и синхронизировать кадры.

Я разобралась, почему важен порядок умножения матриц (сначала масштаб, затем поворот и перенос), чем системы координат Vulkan отличаются от OpenGL, как направление обхода треугольников влияет на отсечение задних граней и как работать с цветом в swarchain формата sRGB. Проверка матриц и геометрии автоматическими тестами помогла находить ошибки, не запуская графику.